GATE CS — Compiler Design

Concise study notes for GATE 2027 · built from the 2016–2026 papers in this repo

Where the marks come from (average marks per paper, 2016–2026)

Topic	Marks / paper	Priority
Parsing (LL/LR, FIRST/FOLLOW)	2.5	Tier 1
Intermediate code, optimisation, data-flow, runtime	1.6	Tier 2
Syntax-directed translation	1.2	Tier 3
Lexical analysis, symbol table, phases	0.5	Tier 3

Compiler Design carries about **5.9 of the 85 technical marks** in a typical paper. Topics below are listed in the order given in the notes; use the table above to decide where to spend time first.

Legend: **Trap** = common mistake · **Tip** = shortcut. For practice, use `GATE_CS_Compiler_Design.pdf` (past questions + new questions with solutions). These notes were written from general knowledge, not checked line by line against a textbook; verify the less common facts.

1. Parsing: FIRST/FOLLOW, LL(1), LR

Phases of a compiler

- Lexical analysis → syntax analysis (parsing) → semantic analysis → intermediate code generation → code optimisation → code generation (+ symbol table and error handling throughout). The first three phases form the analysis (front end); optimisation and generation form the synthesis (back end).

FIRST and FOLLOW

- FIRST(α): set of terminals that can begin a string derived from α (ε included if α can derive ε). Rules: FIRST(a) = {a}; for A → X_1X_2…, add FIRST(X_1) − ε, and continue to X_2 only if X_1 can derive ε.
- FOLLOW(A): terminals that can appear immediately after A in a sentential form. FOLLOW(start) contains $. For B → αAβ: add FIRST(β) − ε; if β can derive ε (or is empty), add FOLLOW(B).
- FOLLOW never contains ε. Compute FIRST first, then FOLLOW iterating until nothing changes.

Top-down parsing (LL)

- Recursive-descent and predictive parsers cannot handle **left recursion** (A → Aα | β). Remove it: A → βA′, A′ → αA′ | ε. **Left factoring** (A → $\alpha\beta_1$ | $\alpha\beta_2$ → A → αA′) removes common prefixes.
- **LL(1) condition:** for every A with productions A → α | β: FIRST(α) ∩ FIRST(β) = ∅; at most one of them derives ε; and if β derives ε then FIRST(α) ∩ FOLLOW(A) = ∅.
- The LL(1) parsing table entry M[A, a] holds A → α if a ∈ FIRST(α), or if ε ∈ FIRST(α) and a ∈ FOLLOW(A). Multiple entries in a cell mean the grammar is not LL(1). Every LL(1) grammar is unambiguous; ambiguous, left-recursive or non-left-factored grammars are not LL(1).
- LL(1) grammars are a proper subset of LR(1) grammars.

Bottom-up parsing (LR, shift-reduce)

- Shift-reduce parsing builds a rightmost derivation in reverse. The **handle** is the substring to reduce. Conflicts: shift/reduce and reduce/reduce.
- Power: LR(0) ⊂ SLR(1) ⊂ LALR(1) ⊂ LR(1). Parser families: LR(0) uses no lookahead; SLR uses FOLLOW sets for reductions; LR(1) (canonical) uses item lookaheads; LALR(1) merges LR(1) states with the same core.
- Number of states: LR(0), SLR and LALR parsers for the same grammar have the same number of states; canonical LR(1) has the same or more. For S → CC, C → cC | d: LR(0)/LALR 7 states, LR(1) 10 states.
- LALR merging can create reduce/reduce conflicts but never new shift/reduce conflicts (relative to LR(1)). SLR is the simplest and most limited; operator-precedence parsers handle only operator grammars (no ε or two adjacent non-terminals).
- Constructing the item sets: closure (add A → .γ for each B → α.Aβ) and goto (move the dot over X). Reduce items (A → α.) go into the action table on lookaheads: FOLLOW(A) for SLR, the item lookahead for LR(1).
- A grammar is LR(0) only if no state has a reduce item together with any other item (shift or reduce) — e.g. S → a is fine but E → E + T | T fails.
- Ambiguous grammars are never LR(k) (yet precedence/associativity rules can resolve conflicts in tools like Yacc).
- Parse-table sizes: rows = states; the action table has columns for terminals and $, the goto table columns for non-terminals.

Trap: LL parsers work from the left and predict productions; LR parsers work from the left but delay decisions until the whole right-hand side is seen. A grammar can be LR(1) but not LL(k) for any k (left recursion).

2. Syntax-Directed Translation

- Attribute grammar: attach attributes to symbols and semantic rules to productions. **Synthesized** attribute: computed from children (or the node's own tokens). **Inherited** attribute: computed from parent and/or left siblings.
- **S-attributed** definitions use only synthesized attributes and are evaluated bottom-up during LR parsing (action at reduction time). **L-attributed**: each inherited attribute depends only on the parent and left siblings, so evaluation works in one left-to-right depth-first pass (also suits LL parsing). Every S-attributed definition is L-attributed.
- Dependency graph: nodes are attributes, edges show which attribute needs which; a valid evaluation order is a topological order (the graph must be acyclic).
- Semantic actions in bottom-up SDT: executed when the production is reduced. The number of actions executed while parsing a string = number of reductions = number of internal nodes in the parse tree. For id + id * id with E → E + T | T, T → T * F | F, F → id: 3 + 2 + 1 + 1 + 1 = 8.
- Evaluating expressions by SDT: `E.val = E₁.val + T.val`. Postfix translation: print the operator after both operands. Constructing syntax trees and DAGs; type checking (type of E_1 + E_2 from the operand types, with coercion rules).
- To trace the output of an SDT with print actions, draw the parse tree and run the actions in the order given by the parser (post-order for actions at the end of the productions).
- Syntax tree vs parse tree: the syntax tree omits unit productions and punctuation.

3. Intermediate Code, Optimisation, Data-Flow, Runtime

Intermediate representations

- Three-address code: at most one operator on the right (`t1 = a + b`). Stored as quadruples (op, arg1, arg2, result), triples (no result field; refer by position), or indirect triples (list of pointers to triples, easy to reorder). Quadruples use more space than triples.
- Number of temporaries and instructions for an expression: one instruction per operator. Reuse of common subexpressions in a DAG reduces instructions.
- Postfix, syntax tree, DAG and three-address code are common IRs; control flow with labels and jumps for if/while.

Basic blocks and control flow graph

- **Leaders:** (1) the first instruction; (2) any instruction that is the target of a jump; (3) any instruction immediately after a jump. A basic block runs from a leader to just before the next leader.
- CFG: nodes are basic blocks, edges are possible control transfers. Dominators, natural loops (back edge whose head dominates its tail), loop-invariant code motion.

Optimisations

- Local: constant folding, constant propagation, copy propagation, algebraic simplification, strength reduction (x*2 → x+x or shift), common subexpression elimination, dead-code elimination, peephole optimisation.
- Loop: code motion, induction-variable elimination, loop unrolling.
- Optimisations must preserve program semantics.

Data-flow analysis

Analysis	Direction	Meet	Typical use
Reaching definitions	forward	union	constant propagation, undefined variables
Live variables	backward	union	register allocation, dead code
Available expressions	forward	intersection	common subexpression elimination
Very busy expressions	backward	intersection	code hoisting

- Equations: `out[B] = gen[B] ∪ (in[B] − kill[B])`; `in[B] = ∪ out[P]` over predecessors (forward, union). Live variables: `in[B] = use[B] ∪ (out[B] − def[B])`, `out[B] = ∪ in[S]` over successors.
- A variable is live at a point if its current value may be used later before being redefined. Number of registers needed ≥ maximum number of simultaneously live variables; register allocation by graph colouring (interference graph).
- Iterate until a fixed point; the order of visiting blocks affects speed, not the answer.

Runtime environment

- Activation record (stack frame): return address, parameters, locals, saved registers, control link (caller's frame), access link (for nested scopes). Stack grows with calls; the heap holds dynamic memory.
- Scoping: static (lexical) vs dynamic. Parameter passing: call by value, reference, copy-restore, name.
- Code generation: instruction selection, register allocation, instruction ordering.

4. Lexical Analysis, Symbol Table

- The lexer reads characters and produces tokens (token name + attribute); a **lexeme** is the actual character sequence; a **pattern** (regular expression) describes the lexemes. It also removes whitespace and comments and tracks line numbers.
- Rules: **longest match (maximal munch)**: `a+++b` → a, ++, +, b (4 tokens); if two patterns match the same length, the one listed first wins (keywords before identifiers).
- Implementation: regular expressions → NFA → DFA; Lex/Flex generates a DFA-driven scanner. Input buffering uses two buffers with sentinels.
- Lexical errors: illegal characters, unterminated strings. Missing semicolon or unbalanced parentheses are **syntax** errors; type mismatch and undeclared variables are **semantic** errors.
- Count tokens: identify keywords, identifiers, operators, constants, punctuation; a string literal is one token; comments are not tokens.
- **Symbol table:** stores names with attributes (type, scope, address). Implemented as a hash table (fast), balanced tree or linear list. Scope handled by a stack of tables or chained tables.
- Lexer and parser errors are detected in different phases; error recovery: panic mode, phrase-level, error productions, global correction.

5. Exam strategy & revision checklist

- Study order: parsing (FIRST/FOLLOW, LL(1) check, LR item sets and conflicts) → data-flow and basic blocks → SDT → lexical/token counting.
- Take care in FOLLOW sets and in listing every LR item; one missed item changes the state count.
- ☐ Compute FIRST/FOLLOW. ☐ Build an LL(1) table. ☐ Build LR(0) and SLR item sets. ☐ Find basic blocks and run a data-flow analysis. ☐ Count tokens. ☐ Redo `GATE_CS_Compiler_Design.pdf` past questions.